

Lab overview

This lab is designed to equip me with practical skills in working with Amazon DynamoDB, a fully managed NoSQL database service that offers scalability, reliability, and performance. The primary focus of this lab is to provide hands-on experience in retrieving data from and populating data to a DynamoDB table by using a Python script. Using DynamoDB can both enhance my understanding of DynamoDB operations and familiarize me with the AWS SDK for Python (Boto3).

Objectives

By the end of this lab, I should be able to do the following:

- Examine the preloaded Python code to understand how it functions through the VS Code integrated development environment (IDE) that I'll use to edit the Python script.
- Review the *LanguagesTable* by using the DynamoDB console and the AWS Command Line Interface (AWS CLI).
- Update the existing Python code so that I can insert an item into the *LanguagesTable*.
- Update the current Python code so that I can query the *LanguagesTable* by using a specific key.
- Test the Python script's overall functionality to update the *LanguagesTable* and read from it.

Task 1: Review the Python script and the LanguagesTable

In this task, I'll connect to a VS Code IDE and review the preloaded Python code to get a better understanding of what it's intended to do. I'll learn about the sections of the code that I must update to add an item to the *LanguagesTable* and to query the table by using a key. Additionally, I'll review the *LanguagesTable* by using both the DynamoDB console and the AWS CLI.

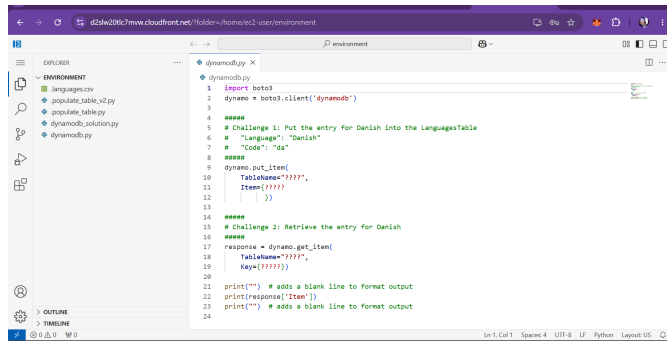
Task 1.1: Accessing the VS Code IDE

In this task, I'll connect to a VS Code IDE that's provisioned as part of this lab and preloaded with the Python script that I am challenged with updating.

Task 1.2: Review the Python script

In this task, I'll review the Python script, and learn about the main sections and what they are intended to do. I'll also identify the sections I am challenged to update.

From the file tree, I'll open the file named [dynamodb.py](#)



```
1 import boto3
2 dynamo = boto3.client('dynamodb')
3
4 #####
5 # Challenge 1: Put the entry for Danish into the LanguagesTable
6 # "Language": "Danish"
7 # "Code": "da"
8 #####
9 dynamo.put_item(
10     TableName="????",
11     Item={????}
12 )
13
14 #####
15 # Challenge 2: Retrieve the entry for Danish
16 #####
17 response = dynamo.get_item(
18     TableName="????",
19     Key={????}
20 )
21 print("") # adds a blank line to format output
22 print(response['Item'])
23 print("") # adds a blank line to format output
```

This Python code uses the boto3 library, which is the AWS SDK for Python, to interact with a DynamoDB database. DynamoDB is a NoSQL database service that AWS provides, and it's known for its low latency and scalability.

The first part of the code (Challenge 1) puts an item into a DynamoDB table. The item represents a language, with *Code* as *da* and *Language* as *Danish*. However, the table name and the specific item details are left as question marks (???), and I am tasked with completing the details.

The second part of the code (Challenge 2), retrieves the item that represents the Danish language from the DynamoDB table. Again, the table name and the key details are left as question marks (???), and I am challenged with filling in those missing details.

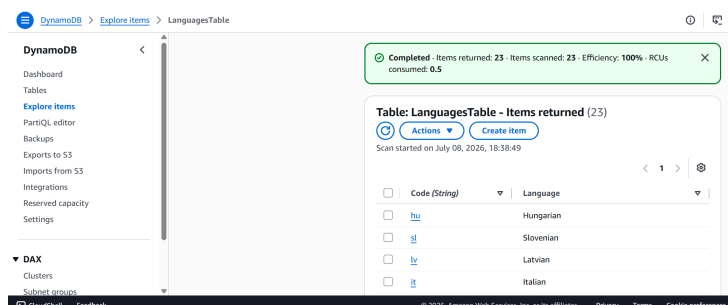
Finally, the code prints the retrieved item. The response from the *get_item* operation is a dictionary, and *Item* is the key in this dictionary that contains the retrieved item.

Task 1.3: Review the LanguagesTable

The *LanguagesTable* has already been created and pre-populated for my use in this lab. It has been loaded with ISO 639-1 codes for languages. In brief, ISO 639 is a set of international standards that lists short codes for language names.

As a developer, it's important to review the *LanguagesTable* to ensure that it exists and that it's populated. Then, when I start testing the updates to the Python script, I can rule out issues with the table being created and populated. The verification of resources applies to development, especially in troubleshooting efforts.

In this task, I'll open the DynamoDB console and review that the *LanguagesTable* exists and is populated.

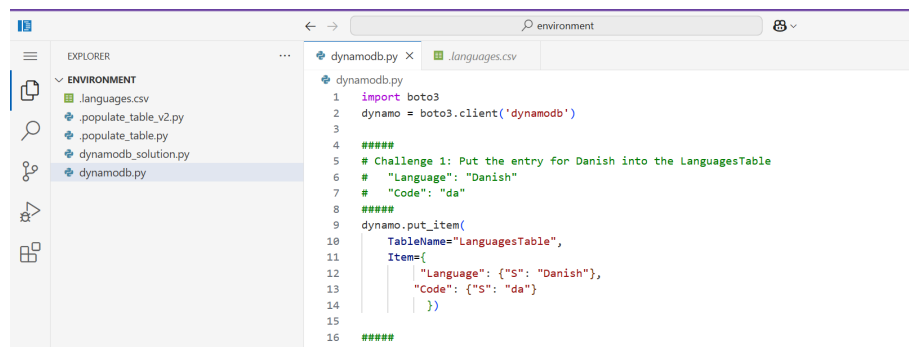


Task complete: successfully accessed the VS Code IDE, which I'll use to modify the existing Python script and add additional functionality. I reviewed the Python script to gain a better understanding of what it's intended to do. I also reviewed the two challenges that I needed to complete so that the Python scripts could read from and write to the *LanguagesTable*. I verified that the *LanguagesTable* exists and that it's populated with ISO 639-1 codes/languages. I completed the verification by using the DynamoDB console.

Challenge 1: Update the Python script to insert an item to the DynamoDB table

In this task, I am challenged with updating the *dynamodb.py* script so that it can *insert* an item into the *LanguagesTable*. This is one of the create, read, update, and delete (CRUD) operations that can be performed on a database.

Based on what I have learned so far about the *LanguagesTable* and its attributes, I'll update the *dynamodb.py* script with the table name and the missing attributes. The script should insert a new language (*Danish*) and its language code (*da*). To update the DynamoDB table with a new entry, I'll use the *put_item* action.

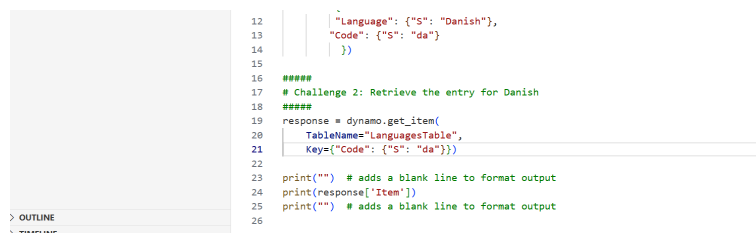


```
1 import boto3
2 dynamo = boto3.client('dynamodb')
3
4 #####
5 # Challenge 1: Put the entry for Danish into the LanguagesTable
6 #   "Language": "Danish"
7 #   "Code": "da"
8 #####
9 dynamo.put_item(
10     TableName="LanguagesTable",
11     Item={
12         "Language": {"S": "Danish"},
13         "Code": {"S": "da"}
14     })
15
16 #####
```

Task complete: I have learned how to use the *put_item* operation to update the Python code in the *dynamodb.py* script. The *put_item* operation adds a new entry in the *LanguagesTable*.

Challenge 2: Modify the Python script to read an item from the DynamoDB table

In this challenge, I am tasked with completing the portion of the script that retrieves the *Code* value for the *Danish* language entry. This is considered to be the *read* operation from the CRUD database operations.



```
12     "Language": {"S": "Danish"},
13     "Code": {"S": "da"}
14 }
15
16 #####
17 # Challenge 2: Retrieve the entry for Danish
18 #####
19 response = dynamo.get_item(
20     TableName="LanguagesTable",
21     Key={"Code": {"S": "da"}})
22
23 print("") # adds a blank line to format output
24 print(response["Item"])
25 print("") # adds a blank line to format output
26
```

Task complete: I have learned how to use the *get_item* operation to update the Python code in the *dynamodb.py* script to read data from *LanguagesTable*.

Task 2: Test the Python script

In this task, I'll test the script to verify that it adds a new entry into the *LanguagesTable* for *Danish* as the *Language*, and *da* as the *Code*. The script should then be able to return the value for the *Code* that's associated with the *Danish* language.

```
{'Language': {'S': 'Danish'}, 'Code': {'S': 'da'}}
```

The output shows the *Language* to be a *string* with a value of *Danish* and the *Code* to be a *string* with a value of *da*, as expected.

Task complete: I have successfully tested my updates to the *dynamodb.py* Python script and verified that it adds *Danish* as a new language and *da* as the language code in the *LanguagesTable*. I also tested the script's ability to query data from the *LanguagesTable*.

Conclusion

I successfully did the following:

- Examined the preloaded Python code to understand how it functions through the VS Code IDE that I used to edit the Python script.
- Reviewed the *LanguagesTable* by using the DynamoDB console and the AWS CLI.
- Updated the existing Python code so that I can insert an item into the *LanguagesTable*.
- Updated the current Python code so that I can query the *LanguagesTable* by using a specific key.
- Tested the Python script's overall functionality to update the *LanguagesTable* and read from it.